Basic Concepts & Commands

Files

- File names can be formed using any character [Alphabets, Numbers, Special characters, Emojis !, ...] ASCII Table
  - Except :

```
‘\’  // Escape character/sequence
‘\0’ // Null Terminator
```

  - Spaces are accepted but **not recommended**, use **dashes** or **underscore** instead.

```
test results.txt → test_results.txt OR test-results.txt
```

- Maximum Length
  - **Per filename:** Usually **255 bytes** (not characters — multibyte characters like `é` or `😊` use more bytes).
  - **Full path length:** Usually **4096 bytes** (including slashes and the terminating null).

> 💡 These limits are defined by the **filesystem** (e.g., ext4, XFS, etc.), not by Linux itself.

- File names are **case sensitive**

```
file.txt, File.txt, and FILE.txt are three different file
s.
```

> 💡 But avoid creating files with same names with difference in case.

- **Leading characters**
  - Filenames starting with `.` are hidden [to protect it] (e.g., `.bashrc`, `.profile`).
  - You can start with `-`, but it can confuse commands (e.g., `rm -file` might be treated as an option). To avoid issues, prefix with `./` when accessing:

```
rm ./-file
```

- **No reserved "names"**
  - Unlike Windows, Linux doesn't reserve names like **CON** or **NUL**.
- **Examples:**

```
# Valid File names
rami?#@.k5
qea*.jk
#$!@r..trm

# Limitations
'/' → This is the directory separator, so it cannot appear
in a filename.
'\0'→ This marks the end of a string in C, so it can't app
ear inside a filename.
```

> 💡 But it is not wise to use special characters in filenames. Example : "-_"

> 💡 Everything else — spaces, tabs, `*`, `?`, `:`, `|`, newlines, even emojis — is technically allowed, though using such characters can be inconvenient when typing commands.

- No concept of file extension, the dot is just another character.

```
file.txt.old
.bashrc
file.doc.mod
.profile.old
results.yesterday
results
```

> 💡 still it is a good idea to stick to the popular extensions for clarity

- **Quiz**

> 💡
>
> ## 🟢 Easy (1–3)
>
> **Q1:** Which character is **not allowed** in a Linux filename?
> a) `*`
> b) `/`
> c) `?`
> d) `#`
> **Q2:** True or False: A file can have spaces in its name in Linux.
> **Q3:** Which of these filenames will be hidden by default?
> a) `myfile.txt`
> b) `.config`
> c) `config.txt`
> d) `-config`
>
> ---
>
> ## 🟡 Medium (4–7)

Q4: What is the **usual maximum number of bytes** for a filename in Linux?
a) 128
b) 255
c) 1024
d) 4096
Q5: True or False: Filenames in Linux are case-insensitive.
Q6: You have three files in the same folder:

```
kotlin
CopyEdit
data.txt
Data.txt
DATA.txt
```

How many of these are considered **different** files by Linux?
Q7: Which command safely removes a file named `-report.txt`?
a) `rm -report.txt`
b) `rm ./-report.txt`
c) `rm -- -report.txt`
d) Both b and c

🔴 Tricky (8–10)

Q8: Can a filename in Linux contain a newline character (`\n`)? (Yes/No)
Q9: You have a file named `my file.txt`. Which of these will open it in `nano`?
a) `nano my file.txt`
b) `nano

Paths

- Directory names are separated by slashes '/'

```
/usr/src/shared/files/myfile.txt
```

- Can be **absolute** or **relative**

- **Absolute**: Does not depend on where you are [Starts from the **root**]

```
/home/ramy/dir
~/dir [Home directory of the current user]
~ is equivalent to /home/aramy
~ramy means /home/ramy (home directory of the user ramy)
```

- **Relative**: Depends where you are in the tree right now

```
./my-project/progress-reports  # Optional in that case

../../my-project/progress-reports

. means current dir

.. means parent dir
```

- General form of commands,

```
username@hostname:/path/....$ <command> [options] [argumen
ts]

# Anything between [] is meant to be optional !
```

The **shell prompt symbol** (like `$`, `#`, `%`, etc.) is useful for several reasons — it's more than just a decoration.

1. Shows that the shell is ready for input

It's the shell's way of saying:

> "I'm ready — type your command now."
>
> Without a prompt, you wouldn't know if the shell is waiting for you or still busy running something.

2. Indicates *who* you are (user vs. root)

By convention:

- `$` → Normal (**non-root**) user
- `#` → Root (**superuser**)

  This helps avoid dangerous mistakes — if you see `#`, you know you have the power to destroy the system accidentally.

Commands

- Commands can be issued on their own without [options] [arguments]

```
$ ls
$ pwd
$ cd
```

- Options usually start with '`-`' or `'--'`
  - `'-'` used with the short option name.
  - `'--'` used with the long option name

```
# Equivalent Commands
$ ls -a
$ ls –all
```

> 💡 We normally use the **short name** in **Command Line Interface (CLI)** and the **long names** in **scripts** (for clarity)

  - If we want to use multiple options sometimes we join options or keep them separate

```
$ ls -a -R
$ ls -aR
$ ls –all –recursive
```

- Sometimes a `'--'` without a name afterwards will mean end of options (even a following hyphen will not be considered a new option)

  Arguments are those info passed to the command such as **file names** and **paths**

  ```
  $ rm -rf ./project-data
  ```

Command	Effect
$ls	**ls**t files
$tree	list a tree of files
$pwd	**p**rint current **w**orking **d**irectory
$cd	**c**hange **d**irectory
$mkdir	**m**a**k**e a **dir**ectory
$cp	**c**o**p**y files and dir
$mv	**m**o**v**e or rename files and dir
$rm	**r**e**m**ove files and directories
$clear	**clear** the screen

$ls (List current Directory)

List is color-coded. [Does not show hidden files by default]

File Type	Color Example	Meaning
Directories	Blue	Folders
Symbolic links	Cyan (Light blue)	Symlinks
Executables	Green	File with execute permission
Archive files	Red	.tar, .zip, .rar, etc.
Image files	Magenta	.jpg, .png, etc.
Devices	Yellow with black	Block or character devices
Sockets	Magenta	IPC socket files
Pipes (FIFOs)	Yellow	Named pipes
Broken symlinks	Red background	Invalid symbolic links

```
$ ls -a (List all files, including hidden files)
$ ls -l (List with long format)

# perm    HL  user  Group size   date      time  dir
drwxr-xr-x 2 me-mac staff 4096 2021-09-02 22:25 Desktop
                          64

# Permissions  [since it's a multiuser system]
- (file)
d (directory)
l (symbolic link)
b (block special file)
c (character special file)
p (named pipe special file)
s (local socket special file)

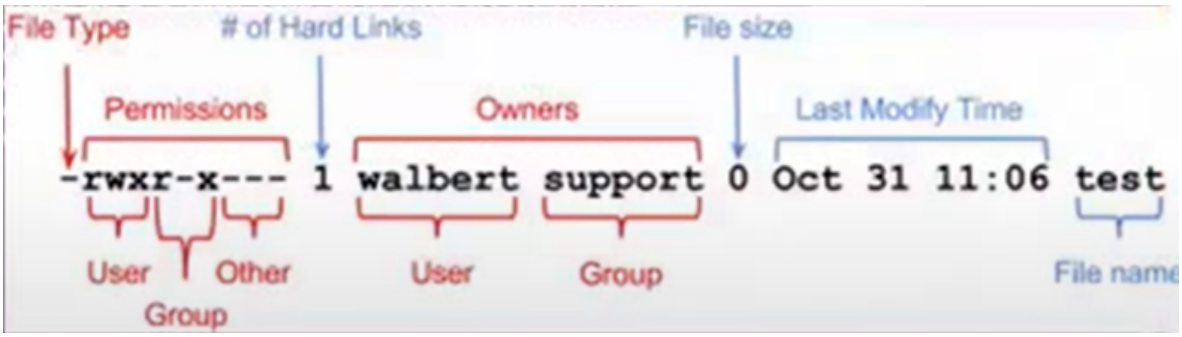
# User, Group, Other [--- --- ---]
r read
w write
x execute

# Number of Hard Links
2

# Owners
User
Group [A member in a group]

# File Size
4096 / 64

# Last Modify Time
2021-09-02 22:25
```



- In **Linux**
  - Most Linux systems use `ext4`, which has a default block size of **4096** bytes (4KB).
  - Even an **empty directory** gets allocated **one full block** (4 KB), which is why you see:
    - 4096
  - That 4096 bytes stores the **internal metadata** for the directory, including **file names** and **inode** numbers.
- In **MacOs**
  - It uses the `APFS` or `HFS+` filesystem.
  - These filesystems can use smaller storage for directory metadata — often showing 64 bytes for an empty directory.
- **Each file has an owner and assigned to a group.**
- **Linux allows users to set permissions on files and directories to protect them.**
- **Permissions are assigned to**
  - **File owner (u)**
  - **Members of the group the file assigned to (g)**
  - **All other users Permissions can only be changed by the owner or root (o)**
- **To display the permissions on files**
  - `ls -l`

- **Linux allows users to set permissions on files and directories to protect them.**
- **Permissions can only be changed by the owner or root.**



Permissions on files and directories		
Permission	Files Access	Directories Access
Read	You can display file contents and copy the file	You can list the directory contents
Write	You can modify the file contents	If you also have execute access, you can add and delete files in the directory
Execute	You can execute the file if it is an executable	You can use the cd command to access the directory

- **To change permissions on a file**

```
chmod
# Symbolic mode
chmod u+x file
chmod g+w file
chmod o-r file

# Numeric (octal) mode
chmod 555 file
chmod 755 file
```

Summary

```
$ ls [options][<dir or file> ..]

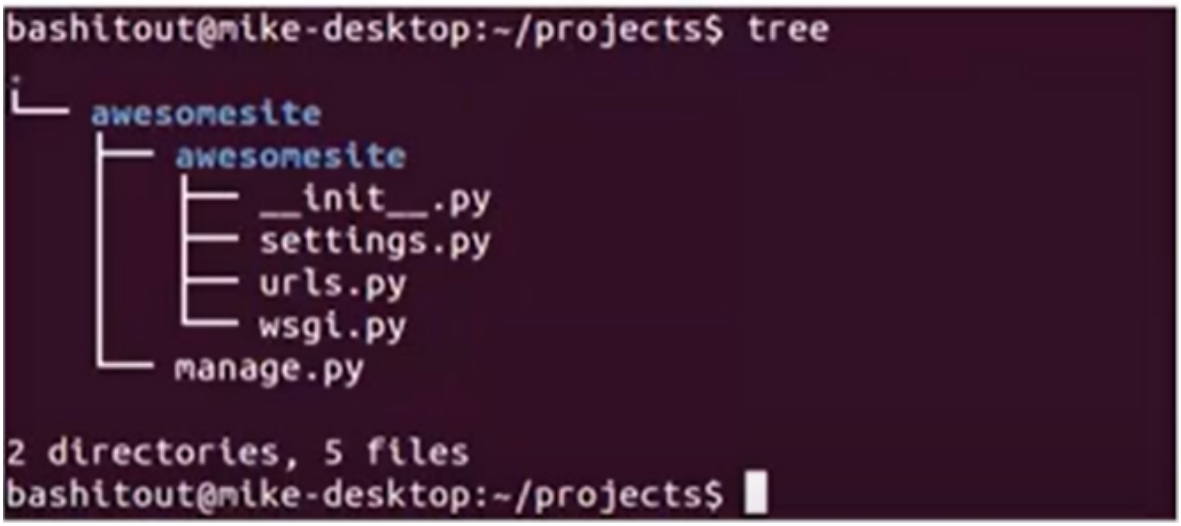
$ls    (list current directory)
$ls -a (all: show hidden files)
$ls -l (long: show file details)
$ls -t (timestamp: sort by descending time modified (most rec
ently modified first))
$ls -r (reverse: make the sort in reverse order)
$ls -S (Size: sort based on file size)

# [You just want to check if a dir exists, or what its permis
sions are, without listing everything inside]
# [Use wildcards to list folders] $ls -d */ shows all subdire
ctories only, not files.
$ls -d (directories: Only show directories)
$ls -R (Recursive: list files inside subdirectories)
$ls <dir> (List the contents of the mentiond directory)
$ls <file> (List the mentioned) [to check the existence or it
s details]
$ls <dir or file> <dir of file> <dir or file> [list selected
dirs of files]
```

Or you can mix these options.

$ tree (List tree from Current Directory)

```
# tree is fully supported in Linux, but it is not installed b
y default on many distros - just like on macOS
$tree
```




```
$tree -a (List tree from Current Directory)
```

```
$tree [options][<dir of file> ..]
$tree
$tree -d (only show directoris)
$tree -a (show all files; including hidden ones)
$tree <dir> (show the tree starting from a different point)
```

$pwd (print current working directory)

```
$pwd
```

$cd [change directory]

```
$cd ~ is equivalent to $cd
$cd - [Switch to the previous working directory]

$cd /etc/netowrk (absolute path)
$cd ../project/ (relatvie path)
$cd project
$cd ~
$cd ~user_name (go to /home/user_name)

$cd .. (go to parent dir)
```

$mkdir [Make directory]

```
$mkdir documents
$mkdir ./documents
$mkdir /home/tom/documents
$mkdir document/text XX lw document msh mwgoed
$mkdir -p documents/text [tmam hy3ml el intermediate subdirec
tories]
$mkdir project1 project2
$mkdir [absolute/relative paths]
```

$cp [Copy]

```
$cp <existing file or dir (source)> <new filename& destinatio
n>
$cp ~/books/science.doc ~/school/science.doc
$cp ~/books/science.doc ~/school/ [default is the src name]
$cp /home/bill/school /home/patric/ XXXX mynf3sh kol user ykt
```

```
b 3nd ltany
#Solution ?
$sudo cp /home/bill/school /home/patrick/ XXX [men lly mn 7a2
o yb2a sudo? fe file config]
#Dirctory !
$sudo cp -r /home/bill/school /home/patrick
```

```
$cp <existing file or dir> <new destination>
$cp file1 file2 (copy file1 to a new file called file2 in the
same location)
$cp file1 ../projects/ (copy file1 to the new location)
$cp -r folder1 ../projects/ (copy the folder with its content
with the same nam)
$cp -r folder1 ../projects/folder2 (copy the folder with new
name)
$cp /etc/passwd . (copy the file ... to here)
$mv <existing file or dir> <new filename & destination>
$mv ~/books/science.doc ~/school/science.doc
$mv ~/books/science.doc ~/school/
$mv /home/bill/school /home/patrick/
$sudo mv /home/bill/school /home/patrick/
$sudo mv -r /home/bill/school /home/patrick/
```

Summary

```
$mv <existing file or dir> <new destination>
```

$rm [Remove]

```
$rm [options] <file or dir list>
```

```
$cd /usr/local/sbin
$rm prog1
```

```
$rm ../etc/file.cfg
$cd ../bin
$rm prog2 prog3
$cd /usr/bin/utils
$rm /usr/bin/utils XXX [ha2of ezay 3leh b3d maytms7]
$cd -
$rm -r /usr/bin/utils
```

Summary

```
$rm [options] <file or dir list>
$rm file1 file2 (remove file1 and file2)
$rm -r ../project2/folder1 (remove the folder with its conten
t)
$rm -i file1 file2 (interactive, ask me before you remove)
$rm -rf ../projects/folder1 (force, force remove)
```

Note:

To remove directories you need always to use ‘-r’

You can not remove your current directory (or any of its parents)